Tema 2

Concurrencia en aplicaciones distribuidas

Introducción

La gestión de concurrencia en sistemas distribuidos constituye uno de los problemas fundamentales de la ingeniería de software y de sistemas. Su objetivo es coordinar múltiples entidades de ejecución -procesos o hilos- que operan simultáneamente y que potencialmente acceden a recursos compartidos, garantizando propiedades de corrección, consistencia y progreso. En sistemas centralizados, este problema ya es complejo; en sistemas distribuidos, la complejidad aumenta debido a la latencia variable, la ausencia de memoria compartida global y la posibilidad de fallos parciales. (Tanenbaum & Van Steen, 2017)

Desde el punto de vista formal, la concurrencia debe preservar:

- **Safety (seguridad):** el sistema nunca entra en un estado inválido.
- **Liveness (vivacidad):** las operaciones eventualmente se completan.
- **Fairness (equidad):** ningún proceso es permanentemente excluido.

En entornos distribuidos, estos objetivos deben alcanzarse bajo un modelo asincrónico donde los mensajes pueden retrasarse indefinidamente y los nodos pueden fallar sin previo aviso (Coulouris et al., 2012). Esto implica que la gestión de concurrencia no puede depender de supuestos fuertes sobre tiempos de respuesta ni sincronización perfecta.

La coordinación puede abordarse desde dos perspectivas:

- Control de acceso a recursos compartidos (exclusión mutua)
- Coordinación de eventos distribuidos (ordenamiento y causalidad)

Ambas dimensiones están profundamente interrelacionadas en aplicaciones reales como sistemas financieros, plataformas de comercio electrónico, redes sociales o sistemas bancarios.

2.1. Concurrencia en aplicaciones distribuidas

En aplicaciones distribuidas, múltiples clientes pueden interactuar simultáneamente con servicios remotos. Por ejemplo, en un sistema bancario distribuido, miles de usuarios pueden intentar modificar el saldo de una cuenta en paralelo. Sin mecanismos adecuados, esto puede producir:

- Condiciones de carrera (race conditions): el resultado depende del orden no determinista de ejecución.
- Actualizaciones perdidas (lost updates).
- Lecturas sucias (dirty reads).
- Inconsistencias temporales entre réplicas.
- Interbloqueos (deadlocks).
- Inanición (starvation).

2.1.1. Modelos de ejecución concurrente

- **Modelo proceso-por-solicitud**

Cada solicitud crea un proceso independiente. Es robusto pero costoso en recursos.

- **Modelo hilo-por-solicitud**

Cada solicitud se atiende con un hilo dentro de un proceso servidor. Es eficiente, pero requiere sincronización interna.

- **Modelo basado en eventos (event loop)**

Un solo hilo maneja múltiples solicitudes usando I/O no bloqueante. Reduce overhead, pero aumenta complejidad de programación.

- **Modelo híbrido (thread pools)**

Combina eficiencia y control de recursos mediante reutilización de hilos.

2.1.2. Exclusión mutua distribuida

En sistemas distribuidos no existe memoria compartida global; por tanto, la exclusión mutua debe implementarse mediante paso de mensajes. Algunos algoritmos clásicos incluyen:

- Algoritmo de Lamport (basado en relojes lógicos)
- Algoritmo de Ricart-Agrawala
- Token Ring distribuido

Estos algoritmos garantizan que solo un proceso acceda a una sección crítica en un momento dado, incluso si los nodos están físicamente separados.

2.1.3. Ordenamiento de eventos y relojes lógicos

La ausencia de un reloj global impide establecer un orden temporal absoluto. Para resolver esto, Leslie Lamport introdujo el concepto de reloj lógico, que permite definir la relación “ocurre antes que” (happens-before) (Lamport, 1978). Este modelo permite:

- Detectar causalidad.
- Implementar exclusión mutua.
- Coordinar eventos distribuidos.

Posteriormente, se desarrollaron relojes vectoriales, que permiten detectar concurrencia real entre eventos.

2.1.4. Control de concurrencia en bases de datos distribuidas

Cuando los datos están replicados, mantener consistencia es un desafío crítico. Se emplean técnicas como:

- Bloqueo en dos fases (2PL)
- Control optimista de concurrencia (OCC)
- Timestamp ordering
- Two-Phase Commit (2PC) para transacciones distribuidas
- Three-Phase Commit (3PC) para mejorar tolerancia a fallos

El protocolo 2PC garantiza atomicidad en sistemas distribuidos, pero introduce bloqueo si el coordinador falla.

En contextos modernos, muchos sistemas adoptan modelos de consistencia eventual, sacrificando consistencia fuerte para mejorar disponibilidad en línea con las implicaciones del Teorema CAP.

2.2. Hilos de ejecución en la programación distribuida

Un hilo de ejecución (thread) es la unidad mínima planificable por el sistema operativo dentro de un proceso. Los hilos comparten el espacio de memoria del proceso, lo que facilita la comunicación, pero requiere mecanismos de sincronización rigurosos.

En aplicaciones distribuidas, los hilos permiten:

- Atender múltiples conexiones simultáneamente.
- Ejecutar operaciones bloqueantes sin detener el sistema completo.
- Aprovechar arquitecturas multinúcleo.
- Implementar paralelismo en servidores de alta demanda.

2.2.1. Modelo de memoria compartida

Dentro de un nodo, los hilos comparten variables globales y heap. Esto introduce problemas como:

- Lecturas inconsistentes
- Escrituras simultáneas
- Problemas de visibilidad de memoria

Para resolverlos se emplean:

- Mutex (Mutual Exclusion Locks)
- Semáforos
- Monitores
- Variables de condición
- Barreras de sincronización

Estos mecanismos aseguran exclusión mutua y orden de acceso.

2.2.2. Pools de hilos

En servidores distribuidos, crear un hilo por cada solicitud puede ser costoso. Por ello se emplean thread pools, que mantienen un conjunto fijo de hilos reutilizables. Esto:

- Reduce overhead de creación.
- Mejora rendimiento.
- Controla consumo de memoria.

2.2.3. Programación asíncrona

Lenguajes modernos incorporan modelos asincrónicos basados en:

- Futures
- Promises
- async/await

Estos mecanismos permiten escribir código no bloqueante, crucial en sistemas de alta concurrencia.

2.2.4. Concurrencia vs. Paralelismo

Concurrencia: múltiples tareas progresan en el mismo intervalo temporal.

Paralelismo: múltiples tareas se ejecutan físicamente al mismo tiempo en distintos núcleos.

En sistemas distribuidos, la concurrencia es intrínseca por naturaleza; el paralelismo depende de la infraestructura subyacente.

2.2.5. Problemas avanzados

Deadlock distribuido: detección mediante grafos de espera

Condiciones de carrera distribuidas

Consistencia de memoria en sistemas multiprocesador

La correcta gestión de hilos en sistemas distribuidos requiere comprender tanto sincronización local como coordinación global entre nodos.

Recursos complementarios

Artículo original de Leslie Lamport-Time, Clocks and the Ordering of Events in a Distributed System

Recurso complementario en línea: https://lamport.azurewebsites.net/pubs/time-clocks.pdf

https://lamport.azurewebsites.net/pubs/time-clocks.pdf

Martin Kleppmann-Material sobre consistencia distribuida

Recurso complementario en línea: https://martin.kleppmann.com/

https://martin.kleppmann.com/

Referencias

Coulouris, G.; Dollimore, J.; Kindberg, T. & Blair, G. (2012). Distributed Systems: Concepts and Design (5th ed.). Addison-Wesley.

Lamport, L. (1978). Time, clocks, and the ordering of events in a distributed system. Communications of the ACM, 21(7), 558–565.

Tanenbaum, A. & Van Steen, M. (2017). Distributed Systems (3rd ed.).

Autoevaluación

1. La propiedad de safety implica que:

a. El sistema no entra en estados inválidos.
b. El sistema siempre responde.
c. La red es rápida.
d. No existen fallos.

2. Una actualización perdida ocurre cuando:

a. Dos procesos escriben concurrentemente y uno sobrescribe al otro.
b. Se elimina una base de datos.
c. Se cae la red.
d. Se usa multihilo.

3. ¿Qué permiten los relojes lógicos?

a. Determinar causalidad entre eventos.
b. Sincronización perfecta.
c. Medir latencia física.
d. Aumentar ancho de banda.

4. El protocolo 2PC garantiza:

a. Atomicidad distribuida.
b. Seguridad de red.
c. Paralelismo.
d. Balanceo de carga.

5. Un deadlock ocurre cuando:

a. Existe espera circular de recursos.
b. Todos los procesos progresan.
c. Hay baja latencia.
d. Se usa async/await.

6. Un thread pool se usa para:

a. Reutilizar hilos y controlar recursos.
b. Aumentar latencia.
c. Crear hilos ilimitados.
d. Eliminar sincronización.

7. En sistemas distribuidos, la comunicación se basa en:

a. Paso de mensajes.
b. Memoria compartida global.
c. BIOS.
d. Disco duro.

8. La inanición ocurre cuando:

a. Un proceso nunca obtiene acceso a recursos.
b. Todos los procesos terminan.
c. Se usa paralelismo.
d. Hay consistencia fuerte.

9. El modelo event-driven se caracteriza por:

a. Un hilo principal con E/S no bloqueante.
b. Crear múltiples procesos.
c. Usar múltiples relojes físicos.
d. Eliminar concurrencia.

10. La concurrencia es inherente en sistemas distribuidos porque:

a. Existen múltiples nodos ejecutando procesos simultáneamente.
b. Todos los nodos son secuenciales.
c. No hay red.
d. No hay datos compartidos.